



Playwright End-to-End Automation Handbook

Modern QA Automation • Real-World • CI/CD Ready

What this covers (End-to-End):

✓ **System setup & configuration**

Playwright install, project initialization, browsers, and essential configs

✓ **Clean project structure**

Tests, fixtures, POM, utilities, data, and scalable folder organization

✓ **Smart locators & navigation**

getByRole, getByLabel, auto-waiting, assertions, and stability best practices

✓ **Core UI automation**

Inputs, dropdowns, radios, checkboxes, tables, collections

✓ **Advanced UI handling**

New tabs & windows, popups, alerts, dialogs, frames & iFrames

✓ **File operations**

File upload & download handling with real-world examples

✓ **Network interception & mocking**

API stubbing, response validation, and flaky-test elimination

✓ **API testing with Playwright**

Backend validation using request (UI-independent & fast)

✓ **Authentication & storage state**

Session reuse, login bypass, and faster test execution

✓ **Fixtures & hooks**

Reusable test setup, environment configs, and secure credentials



✓ **Data-driven testing**

Multiple test scenarios using arrays, configs, and external data

✓ **Page Object Model (POM)**

Maintainable, readable, and enterprise-ready test design

✓ **Debugging superpowers**

Screenshots, videos, traces, Playwright UI mode

✓ **Parallel execution & retries**

Faster pipelines with smart concurrency strategies

✓ **Reporting & CI/CD**

HTML reports, GitHub Actions, Azure DevOps pipelines

✓ **Best practices from real projects**

Stable tests, clean code, flake reduction, team-friendly automation

 Goal:

Help QA engineers understand **how a real, production-ready Playwright automation framework is designed, implemented, and scaled** — not just how individual features work.

This handbook bridges the gap between:

“I know Playwright” → “I can build enterprise-level automation.” 

 Creator: Adem Gokce

 *Sharing is caring. Learning is a lifelong passion.*

 Please feel free to share this with colleagues, students, or anyone learning QA automation — **from beginner to advanced level.**

 Happy Testing |  Think like a user, automate like a pro

☀️ Why This Handbook?

Playwright is fast, reliable, and built for real-world end-to-end testing. This handbook is designed to:

- Help **manual QAs transition to automation**
- Give **ready-to-copy examples** 🧩
- Share **best practices used in real projects**
- Be **CI/CD-ready** from day one

If you're a QA who wants **confidence + clarity**, this is for you 💪

📁 Agenda (Learning Flow That Makes Sense)

🔧 Foundations

- Setup & Configuration
- Test Structure & Project Layout

⚙️ Core Playwright Skills

- Locators, Navigation & Auto-Waiting
- Inputs, Dropdowns, Radios & Checkboxes
- Tables & Collections

🗉 Advanced UI Handling

- New Tabs / Windows
- Alerts & Dialogs
- Frames & iFrames
- File Upload & Download

🌐 Backend Power

- Network Interception & Mocking
- API Testing with request

- Authentication & Storage State

Scalable Framework Design

- Fixtures & Hooks
- Data-Driven Testing
- Page Object Model (POM)

Debugging, Reporting & CI

- Screenshots, Videos & Traces
- Parallel Execution & Retries
- GitHub Actions & Azure DevOps

Quick Setup (5 Minutes)

Install Playwright

```
npm init -y
```

```
npm i -D @playwright/test
```

```
npx playwright install
```

Run Tests

```
npx playwright test
```

```
npx playwright test --ui
```

 **Tip:** --ui is perfect for beginners to visually debug tests 🙄

playwright.config.ts (Essentials)

```
import { defineConfig } from '@playwright/test';
```

```
export default defineConfig({
```

```
  testDir: './tests',
```

```
  fullyParallel: true,
```

```
retries: 1,  
use: {  
  baseURL: 'https://rahulshettyacademy.com',  
  headless: true,  
  trace: 'on-first-retry',  
  screenshot: 'only-on-failure',  
  video: 'retain-on-failure'  
}  
});
```

📌 **Why this matters:** You get debugging artifacts only when needed → faster pipelines.

🎯 Locators & Navigation (Golden Rule)

✅ Prefer User-Facing Locators

```
page.getByRole('button', { name: 'Sign In' });  
page.getByPlaceholder('Username');  
page.getByLabel('Password');
```

💡 These survive UI changes better than CSS/XPath.

🌐 Navigation with Auto-Waiting

```
await page.goto('/loginpagePractise/');
```

Playwright automatically waits — **no Thread.sleep needed** 🚫

📝 Inputs, Dropdowns, Radios & Checkboxes

```
import { test, expect } from '@playwright/test';  
test('Basic controls', async ({ page }) => {
```

```
await page.goto('/loginpagePractise/');
await page.getByPlaceholder('Username').fill('rahulshettyacademy');
await page.getByPlaceholder('Password').fill('learning');
await page.locator('select').selectOption('consult');
await page.getByLabel('User').check();
await expect(page.getByRole('button', { name: 'Sign In' })).toBeEnabled();
});
```

💡 **Think like a user, automate like a pro.**

Child Windows / New Tabs

```
test('Popup handling', async ({ page }) => {
  await page.goto('/loginpagePractise/');
  const link = page.getByRole('link', { name: '/documents request/i' });
  const [popup] = await Promise.all([
    page.waitForEvent('popup'),
    link.click()
  ]);
  await expect(popup).toHaveTitle('/documents/i');
});
```

💡 **Clean, readable, and reliable.**

File Upload & Download

Upload

```
await page.getByLabel('Upload').setInputFiles('fixtures/sample.pdf');
```

Download

```
const [download] = await Promise.all([
  page.waitForEvent('download'),
  page.getByRole('button', { name: 'Download' }).click()
]);
```

Frames & iFrames

```
const frame = page.frame({ name: 'frame-name' });
await frame?.getByRole('button', { name: 'Submit' }).click();
```

Or cleaner:

```
page.frameLocator('#frameId').getByText('Hello').click();
```

Alerts / Dialogs

```
page.on('dialog', d => d.accept());
await page.getByRole('button', { name: 'Trigger Alert' }).click();
```

 Simple and powerful.

Tables & Collections

```
const rows = page.locator('table tbody tr');
for (let i = 0; i < await rows.count(); i++) {
  const name = await rows.nth(i).locator('td').first().textContent();
  if (name?.includes('Nike')) {
    await rows.nth(i).getByRole('button', { name: 'Add' }).click();
  }
}
```

 Real-world automation skill.

Network Interception & Mocking

```
await page.route('**/api/products', route => {  
  route.fulfill({  
    status: 200,  
    json: { items: [{ id: 1, name: 'Mocked Shoe' }] }  
  });  
});
```

 Kill flaky dependencies.

API Testing (No UI Needed)

```
import { test, expect, request } from '@playwright/test';  
  
test('API create & fetch', async () => {  
  const ctx = await request.newContext({ baseURL: 'https://reqres.in' });  
  const res = await ctx.post('/api/users', { data: { name: 'QA', job: 'tester' } });  
  expect(res.ok()).toBeTruthy();  
});
```

 Fast feedback.

Authentication & Storage State

```
test('save login state', async ({ page }) => {  
  await page.goto('/login');  
  
  // login steps  
  
  await page.context().storageState({ path: 'state.json' });  
});
```


Reuse it in config → ⚡ faster tests.

Fixtures & Hooks

```
import { test as base } from '@playwright/test';

type Fixtures = { creds: { user: string; pass: string } };

export const test = base.extend<Fixtures>({

  creds: async ({}, use) => {

    await use({ user: process.env.USER!, pass: process.env.PASS! });

  }

});
```

Data-Driven Testing

```
const cases = [

  { user: 'u1', pass: 'p1' },

  { user: 'u2', pass: 'p2' }

];

for (const c of cases) {

  test(`login for ${c.user}`, async ({ page }) => {

    await page.goto('/login');

    await page.getByPlaceholder('Username').fill(c.user);

  });

}
```

Page Object Model (POM)

```
export class LoginPage {
```

```
constructor(private page) {}  
  
user = () => this.page.getByPlaceholder('Username');  
pass = () => this.page.getByPlaceholder('Password');  
  
async goto() {  
  await this.page.goto('/loginpagePractise/');  
}  
}
```

🧠 Clean tests. Happy teams.

🐞 Screenshots, Videos & Traces

```
await page.screenshot({ path: 'screens/home.png', fullPage: true });
```

After run:

```
npx playwright show-report
```

⚡ Parallelism & Retries

```
fullyParallel: true,
```

```
retries: 1,
```

```
workers: '50%'
```

Override anytime via CLI.

🚀 CI/CD (GitHub Actions)

- uses: actions/checkout@v4
- uses: actions/setup-node@v4
- run: npm ci
- run: npx playwright install --with-deps

